



Lecturer,
Dept. of CSE, ULAB, Dhaka



EMAIL:
atanu.shuvam@ulab.edu.bd

CSE 2201 : Algorithms

Lecture 02: Linear Data Structures

Atanu Shuvam Roy



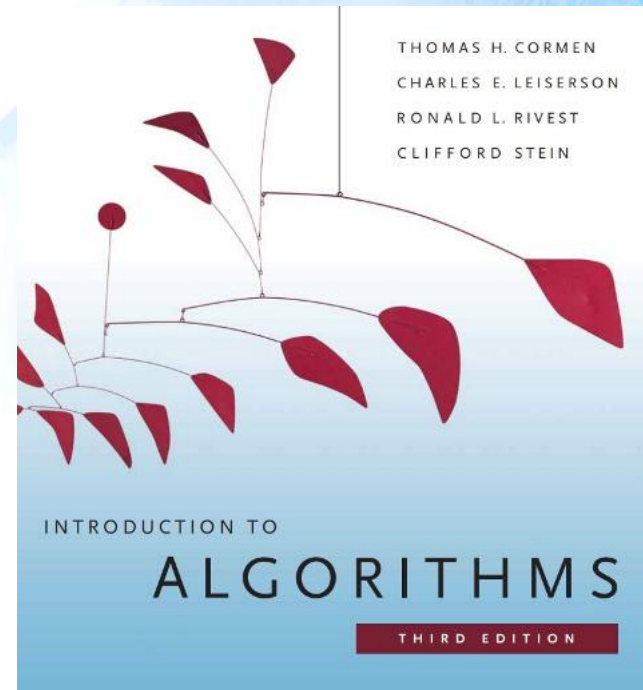
Class Code
GVWTHYJV



Fall 2025

Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



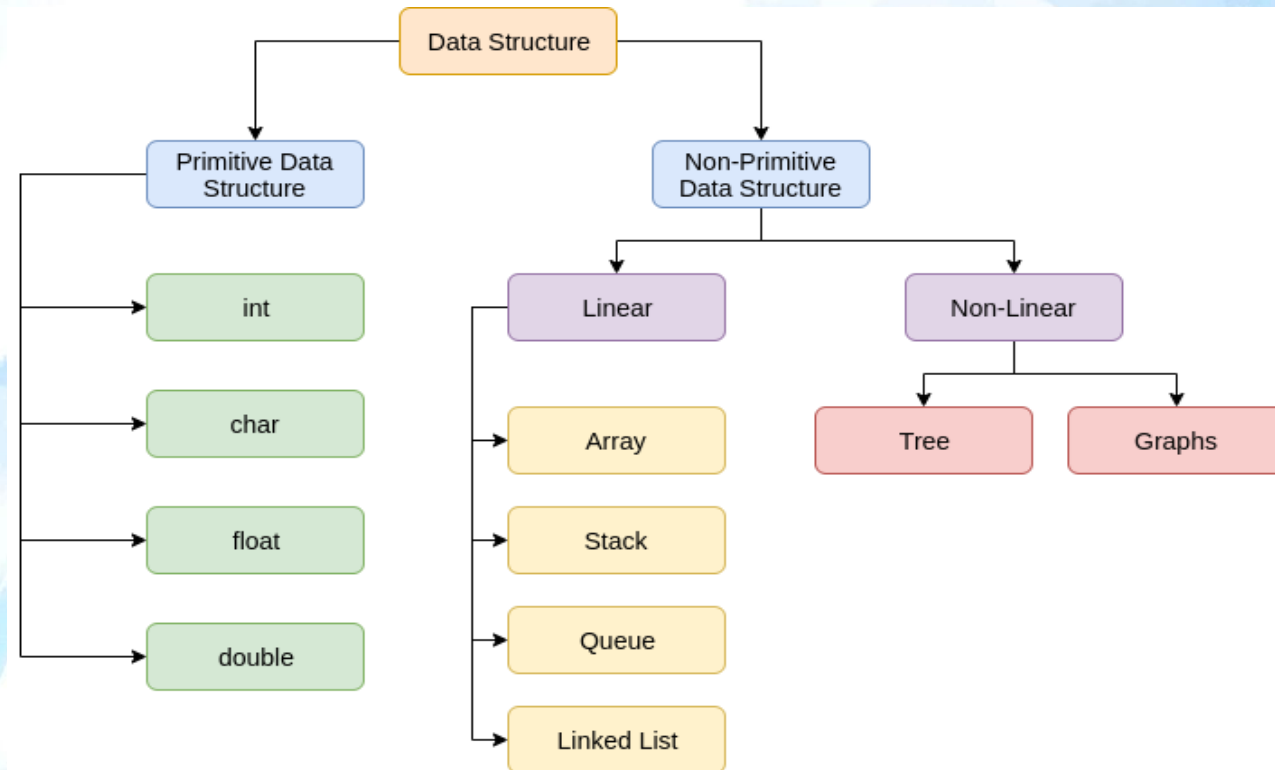
Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100

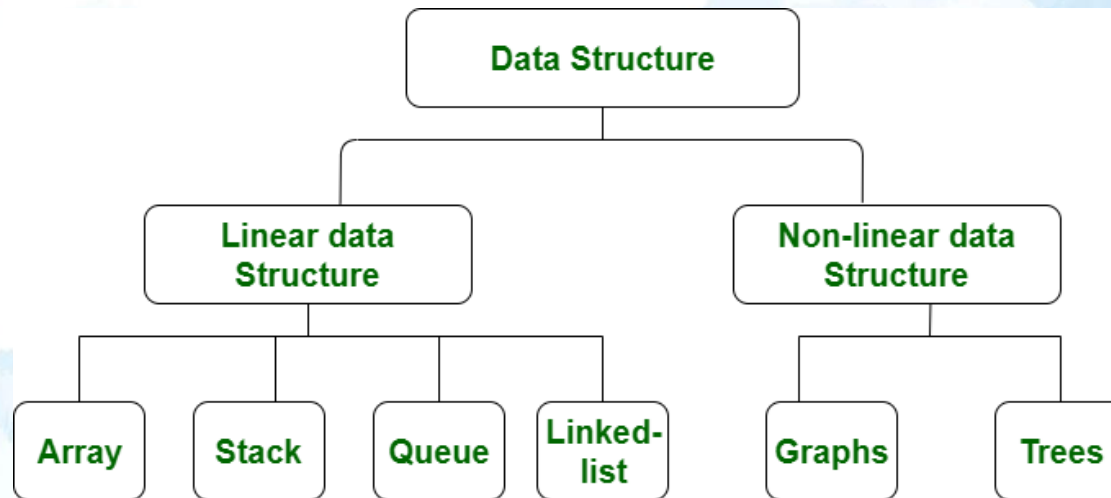
Today's Contents

- **Different types of Data Structures**
- **What is a Linear Data Structures?**
- **Different types of Linear Data Structures**
- **Array**
- **Some exercises**

Different types of Data Structure



Different types of Data Structure



The data structures can also be classified on the basis of the following **characteristics**:

Characteristic	Description
1. Linear	The data items are arranged in a linear sequence. Example: Array, Linked List
2. Non-Linear	The data items are not in sequence. Example: Tree, Graph
3. Homogeneous	All the elements are of same type. Example: Array
4. Non-Homogeneous	The elements may or may not be of the same type. Example: Structures, Linked List
5. Static	Sizes and structures associated memory locations are fixed. Example: Array
6. Dynamic	Expands or shrinks depending upon the program need and its execution. Also, their associated memory locations changes. Example: Linked List

Linear Data Structure

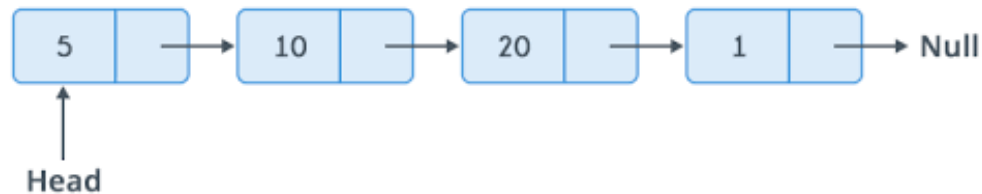
- In Linear data structures, the **data items are arranged in a linear sequence.**
- The common examples of the **linear data structure** are:
 - **Arrays**
 - **Linked lists**
 - **Stacks**
 - **Queues**

Example of Linear Data Structure

Array

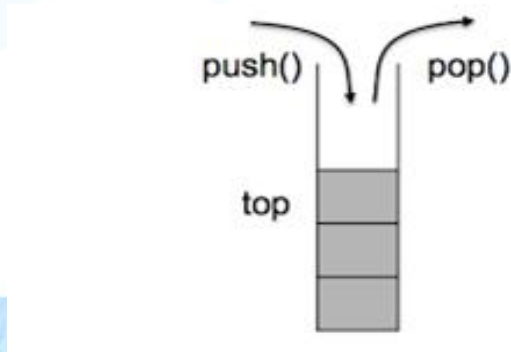
10	20	30	40	50	60	70			
----	----	----	----	----	----	----	--	--	--

Linked List



Example of Linear Data Structure

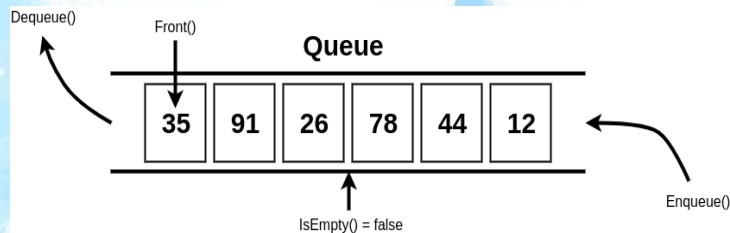
Stack



EXAMPLES OF STACK:



Queue



Linear Data Structure

- There are **two techniques** of representing such linear structure within memory.

The first way is to provide the linear relationships among all the elements represented **using linear memory location**. These linear structures are termed as **arrays**.

10	20	30	40	50	60	70			
----	----	----	----	----	----	----	--	--	--

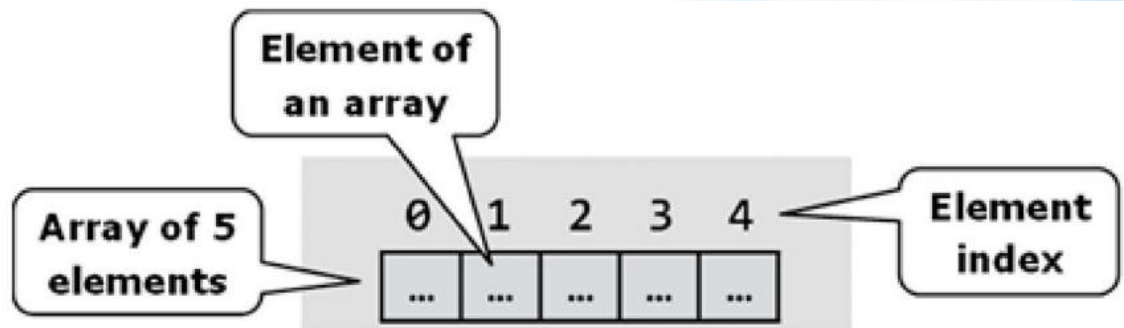
The second technique is to provide a linear relationship among all the elements represented by **using the concept of pointers or links**. These linear structures are termed as linked lists.



Review of Some Previous Topics

- **Array**
- **Structures**
- **Function**
- **Pointer**

Linear Data Structure – 1: Array



We will know

- **Basic Concepts of Array**
- **Array Representation**
- **Basic Operations we perform**
- **Advantages**
- **Limitations**

Data Structure – Array

- The array is a **fixed-size sequenced collection of data** belonging to the **same data types**.
- Stores data in **contiguous memory locations**.
- An array is a collection of **homogeneous (same type) data** items

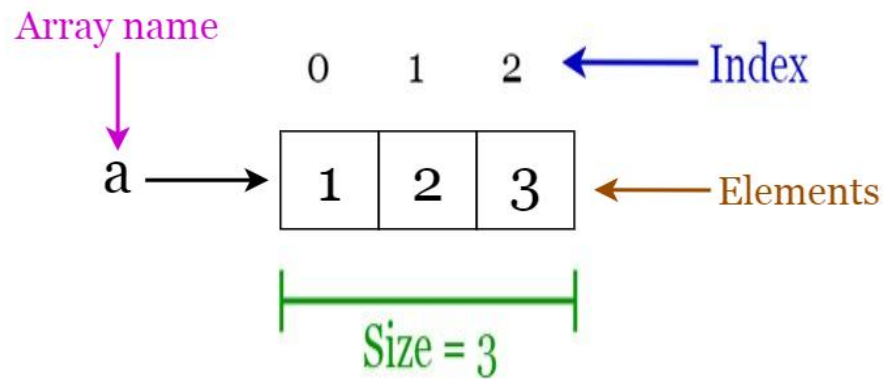
For example if an array is of type “**int**”, it can only store integer elements

and cannot allow the elements of other types such as double,

float, char etc.

- The **Size** of the array is **Fixed**.

Array Representation



An array stores data at contiguous memory locations

Integer type Array

Memory Location											
200	204	208	212	216	220	224	228	232	236	240	244
11	9	17	89	1	90	19	5	3	23	43	99
0	1	2	3	4	5	6	7	8	9	10	11
Index											

Characher type Array

Memory Location									
200	201	202	203	204	205	206	▪	▪	▪
U	B	F	D	A	E	C	▪	▪	▪
0	1	2	3	4	5	6	▪	▪	▪
Index									

The syntax for declaring array

data_type **array_name** [**array_size**];

If we want to declare an array to store **10 integer numbers**, then we have to write:

int arr [10];

0	1	2	3	4	5	6	7	8	9

Basic Operations of Array

1. Insert	– insert item in an array.
2. Search	– search an item in an array.
3. Traverse	– visit the elements and print them.
4. Update	– update an existing item in an array.
5. Delete	– delete an existing item from an array

1. Insert

– insert item in an array

When we write: **int marks [10];**

0	1	2	3	4	5	6	7	8	9

Storing data in the array

```
for(int i = 0; i < 5; i++) {  
    marks[i] = 100;  
}
```

1. Insert

– insert item in an array

When we write: **int marks [10];**

0	1	2	3	4	5	6	7	8	9
100	100	100	100	100					

Storing data in the array

```
for(int i = 0; i < 5; i++) {  
    marks[i] = 100;  
}
```

1. Insert

– insert item in an array

When we write: **int marks [10];**

0	1	2	3	4	5	6	7	8	9

Storing data in the array

```
for(int i = 0; i < 7; i++ ) {  
    marks[i] = 100+i;  
}
```

1. Insert

– insert item in an array

When we write: **int marks [10];**

0	1	2	3	4	5	6	7	8	9

taking input and storing it in an array

```
for(int i = 0; i < 6; i++ ) {  
    scanf("%d", &marks[i]);  
}
```

2. Search – search an item in an array.

Search a data from the array:

Search, X = 50

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

Pseudocode of Seaching in an array

Search, X = 50

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

```
found = 0;
```

```
for( i = 0; i < n; i++ )  
{  
    if ( marks [i] == X )  
    {  
        found = 1;  
        break;  
    }  
}
```

```
if(found == 1)  
{  
    Data Found  
}  
else  
{  
    Data Not Found  
}
```

3. Traverse – visit the elements and print them.

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

- **Print** the elements of the Array
- **Count** the elements
- Find the **sum/average** of the elements

4. Update – update an existing item in an array.

Update the value of **Index = 2** with new data = **555**

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

marks[2] = 555

0	1	2	3	4	5	6	7	8	9
10	20	555	40	50	60	70			

5. Delete – delete an existing item from an array

Delete the value from array.

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

After **deleting the Last data** from the array

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60				

Total number of elements will be **(n-1)**

Advantages of Array

- **Useful** when we are dealing with lot of variables of the **same type**.
- **Very easy to implement.**
- In arrays, the elements can be **accessed randomly** by using the **index number**.
- **Efficient** in terms of **Time Complexity and Space Complexity**.

Limitations of Array

1. Array is a **static** Data Structure: size is **Fixed**
1. **Insertion** and **deletion** are **quite difficult**

Limitations of Array

1. While using array, we must need to make the decision of the size of the array in the beginning, so if we are not aware how many elements we are going to store in array, it would make the task difficult.
1. The size of the array is fixed so if at later point, if we need to store more elements in it then it can't be done. On the other hand, if we store less number of elements than the declared size, the remaining allocated memory is wasted.
1. Insertion and deletion are quite difficult in an array as the elements are stored in consecutive memory locations and the shifting operation is costly.

Limitation: Insert item in an **specific Index**

Insert new value, **555** in Index **2**

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

The array after **inserting new value, 555** in Index **2**

0	1	2	3	4	5	6	7	8	9
10	20	555	30	40	50	60	70		

Write the pseudocode to insert new data in a specific index

new value = 555

index 2

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

Write the pseudocode to insert new data in a specific index

new value = 555
index 2

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

0	1	2	3	4	5	6	7	8	9
10	20	555	30	40	50	60	70		

```
new_value = 555  
index = 2
```

```
for ( i = n-1 ; i>=2 ; i--)  
{  
    marks [i+1] = marks[i];  
}
```

```
marks [2] = 555;
```

Limitation: Insert item in an **specific Index**

Insert new value, 555 in Index 2

0	1	2	3	4	5	6
10	20	30	40	50	60	70

The array after inserting new value, 555 in Index 2

0	1	2	3	4	5	6
10	20	555	30	40	50	60

Limitation: Delete an item from a **specific Index**

Delete value from index 2

0	1	2	3	4	5	6	7	8	9
10	20	555	30	40	50	60	70		

The array after Deleting data from Index 2

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70			

Write the pseudocode to Deleting data from Index 2

Index = 2

0	1	2	3	4	5	6	7	8	9
10	20	555	30	40	50	60	70		

Home Task

1. Declare an array of size 100, take integer numbers as input. Display the Elements.
2. Find out all even numbers from the above array.
3. Search a data from the array.
4. Insert an element in given index.
5. Delete data from given index

References

https://www.tutorialspoint.com/data_structures_algorithms/array_data_structure.htm

<https://www.javatpoint.com/data-structure-array>

<https://beginnersbook.com/2018/10/data-structure-array/>

References-2

1. <https://www.geeksforgeeks.org/data-structures/>
2. https://www.tutorialspoint.com/data_structures_algorithms/data_structures_basics.htm
3. <https://www.programiz.com/dsa>
4. <https://www.javatpoint.com/data-structure-tutorial>
5. <https://www.w3schools.in/data-structures-tutorial/intro/>
6. <https://towardsdatascience.com/8-common-data-structures-every-programmer-must-know-171acf6a1a42>

Original Slides Taken from Pramanik Sir



ULAB

UNIVERSITY OF LIBERAL ARTS
BANGLADESH

Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh
Department of Computer Science & Engineering